Lecture 47 : Project Monitoring and Control (Contd.)

Introduction

In the dynamic world of software development, projects are rarely static. Requirements change, bugs are discovered, and new features are constantly added, leading to a continuous evolution of the software product. Without a robust system to manage these changes, a project can quickly descend into chaos, leading to inconsistencies, lost work, and significant delays. This lecture delves into the critical domain of software configuration management (SCM), a discipline essential for maintaining order and integrity throughout the software lifecycle.

Effective SCM ensures that all modifications to a software system are systematically tracked, controlled, and documented. It provides the necessary framework for teams to collaborate efficiently, prevent unauthorized alterations, and ensure that the right versions of components are always available. By understanding the processes and tools involved in SCM, developers and project managers can navigate the complexities of software evolution with confidence, delivering high-quality products on time and within budget. This session will explore the core activities of configuration management, the mechanisms for controlling changes, and introduce some of the prevalent tools that facilitate this crucial aspect of project success.

Core Concepts

Term	Definition	Significance
`Configuration Management Process`	A systematic approach to managing changes to a software system throughout its lifecycle.	Ensures order, consistency, and traceability of all software artifacts, preventing chaos.
`Configuration Identification`	The process of determining which system components should be placed under configuration management.	Establishes the scope of control, identifying critical items like code, documents, and test plans.
`Configuration Control`	The mechanism used to ensure that all changes made to identified system	Prevents unauthorized modifications and ensures that changes are reviewed

Term	Definition	Significance
	components are authorized and occur smoothly.	and approved before integration.
`Controlled Work Products`	Software artifacts that are formally placed under configuration control, requiring specific procedures for modification.	Guarantees the integrity and stability of critical project components, such as source code and design documents.
`Pre-controlled Work Products`	Items not yet under formal configuration control but are expected to be in the near future.	Allows for initial flexibility while anticipating future formal management, aiding in planning.
`Uncontrolled Work Products`	Software artifacts that will never be subject to formal configuration control.	Differentiates between critical and non-critical items, focusing management efforts where they are most needed.
`Reserve Operation`	A developer's request to obtain a private, modifiable copy of a controlled work product.	Prevents concurrent modification issues by granting exclusive temporary access to a component.
`Restore Operation`	The process of replacing the master copy of a work product with a modified, approved private copy.	Integrates validated changes back into the main system, updating the official version.
`Change Control Board (CCB)`	A group, often comprising project managers or senior team members, responsible for reviewing and approving proposed changes.	Acts as a gatekeeper for modifications, ensuring changes are justified, documented, and validated.
`Deltas`	The specific changes or modifications required to transform one version of a	An efficient storage method used by tools like SCCS/RCS to save disk space by

Term	Definition	Significance
	software component into the next.	storing only differences, not full versions.
`Release Management`	The process of systematizing how new software releases are provided by developers and obtained/used by users.	Streamlines the deployment of new versions, minimizing effort for both development teams and end-users.

Detailed Analysis

1. The Configuration Management Process

Software configuration management (SCM) is fundamentally carried out through two principal activities: configuration identification and configuration control. These activities work in tandem to ensure that all aspects of a software project are managed effectively throughout its lifecycle.

Configuration Identification
This initial phase involves deciding which parts of the system are critical enough to be placed under configuration management. Project managers classify work products into three main categories:
* **Controlled Work Products**: These are items that are formally placed under configuration control. Any changes to these products require adherence to specific, formal procedures. Examples include requirement specification documents (SRS), design documents (DFDs, UML diagrams), source code for each module, test cases, test plans, and problem reports. Even tools used in the system, such as compilers, linkers, and parsers, can be considered controlled work products.
* **Pre-controlled Work Products**: These are items not yet under configuration control but are anticipated to be in the near future. This category allows for a phased approach to control, where items gain formal management as they mature or become more critical.
* **Uncontrolled Work Products**: These are items that will not be subject to configuration control. They are typically less critical or temporary artifacts that do not require formal tracking of changes.
The combination of controlled and pre-controlled work products is often referred to collectively as "controllable work products."

Configuration Control
This process is central to SCM, directly impacting the day-to-day operations of developers. Its primary goal is to ensure that only authorized changes are made to

controlled work products, thereby preventing unauthorized modifications.
* **Authorization**: Configuration control allows changes only by authorized personnel, such as specific team members granted permission by the project manager or a steering committee. This prevents arbitrary alterations to critical components.
* **Prevention of Unauthorized Changes**: By restricting modification rights, the system ensures that any developer wishing to alter a controlled work product must first obtain permission. This gatekeeping function is crucial for maintaining system integrity.

2. Configuration Control Workflow

The process of making changes to a controlled work product follows a structured workflow, typically involving "reserve" and "restore" operations, and oversight by a Change Control Board (CCB).

- **Reserve Operation**: When a developer needs to modify a controlled work product (e.g., a code module), they initiate a "reserve" request. This request is sent to the project manager or steering committee. If approved, the developer is provided with a private copy of the work product in their local workspace. This mechanism prevents multiple developers from simultaneously modifying the same master copy, thereby avoiding concurrency issues. Configuration management tools are designed to allow only one team member to reserve a module at any given time, blocking others until the reserved module is restored.
- **Modification**: The developer then makes the necessary changes to this private copy. It is crucial that changes are made only to the private copy, not directly to the master copy.
- **Change Control Board (CCB) Review**: Once the developer has completed and tested their changes, the modified private copy is submitted for review. This review is typically conducted by a Change Control Board (CCB). For smaller projects, the project manager might fulfill the role of the CCB. For larger projects, the CCB is a dedicated group, often composed of senior development team members.
  The CCB's responsibilities include:
- **Reviewing Motivation**: Ensuring the change is well-motivated and justified.
- **Assessing Impact**: Verifying that the developer has considered and documented the potential effects (positive or negative) of the change.
- **Checking Interoperability**: Confirming that the changes integrate well with modifications made by other developers.

- **Validating Changes**: Ensuring that appropriate personnel have validated and tested the change (e.g., testing changed code for consistency with requirements).
- **Restore Operation**: If the CCB approves the changes, the project manager authorizes the "restore" operation. This operation replaces the old master copy of the work product in the configuration management repository with the newly modified and approved private copy. Configuration tools enforce this by not allowing a developer to replace a work product with their local copy without explicit authorization from the CCB. This ensures that only completely, properly, and appropriately modified work products are updated in the official configuration.

3. Release Management

Release management is a critical process that systematizes how new software releases are provided by developers and how users smoothly obtain and utilize these new versions. Its importance has grown significantly with the ease of software distribution via the internet.

The core objectives of release management are:
* **Minimal Developer Effort**: The process should require minimal effort from developers to prepare and upload a new software release.
* **Minimal User Effort**: Users should be able to download and install new releases with minimal difficulty.
* **Automated Component Identification**: When a new version is to be released, the system should automatically identify changed components, their dependencies, and retrieve all interdependent components as a group to prevent inconsistencies.
* **Avoidance of Unnecessary Retrievals**: The system should avoid retrieving unnecessary or unchanged components, optimizing the release package and download size.

4. Configuration Management Tools: SCCS and RCS

For managing software configurations, particularly in UNIX environments, two popular open-source tools are the Source Code Control System (SCCS) and the Revision Control System (RCS).

- **Functionality**: Both SCCS and RCS are designed for controlling and managing different versions of text files. A key limitation is that they do not handle binary files (e.g., executable files, images, diagrams).
- **Efficient Storage using Deltas**: To minimize disk space, SCCS and RCS do not store full copies of every version. Instead, they store the original,

baseline version (e.g., module 1.1) along with the deltas—the specific changes needed to transform the baseline into subsequent versions (e.g., changes from 1.1 to 1.2, and from 1.2 to 1.3). This approach significantly reduces storage requirements compared to storing each full revision.

- **Access Control**: These tools provide facilities for checking components "in" and "out."
- **Check-out**: Individual developers can "check out" components, which is analogous to the "reserve" operation, allowing them to modify a private copy.
- **Check-in**: After modifications are made, reviewed, and approved (often by a CCB), developers can "check in" the changed module. This replaces the master file with the updated version.
- **Version Numbering and Variants**: Revisions are typically numbered in ascending order (e.g., 1.1, 1.2, 1.3). SCCS also supports creating variants of a component by "forking" the development history, allowing for parallel development paths.

5. Other Project Management Tools

Beyond specific configuration management, a range of tools exists to support broader project management activities, such as scheduling, resource allocation, and cost tracking.

- **Gantt Project**: This is a freeware, GPL-licensed, open-source project management software compatible with Windows, Linux, and macOS. It is particularly strong in:
- **Scheduling**: Supports project scheduling, including techniques like CPM and PERT.
- **Resource Management**: Excellent for representing and managing project resources.
- **Microsoft Project**: A widely used commercial project management software from Microsoft Corporation. Its capabilities include:
- **Portfolio Management**: Supports managing a collection of projects.
- **Web-based Access**: Offers web-based functionalities.
- **Scheduling**: Facilitates scheduling activities using CPM and PERT.
- **Cost Management**: Provides tools for tracking and managing project costs.
- **Resource Management**: Supports the allocation and tracking of resources. Advanced capabilities are often supported through Microsoft Office Project Server and Microsoft Office Project Web Access software.

- **Primavera Project Management Software**: A comprehensive suite of project management tools, often used for large-scale projects. It offers different versions:
- **SureTrack**: An entry-level software, which allows project managers to define phases, establish goals, anticipate obstacles, and delegate assignments.
- **Primavera 6**: An advanced version with extensive features.
  Like Microsoft Project, Primavera is a commercial software and provides:
- **Portfolio Management**: For overseeing multiple projects.
- **Web-based Access**: For remote collaboration and access.
- **Scheduling**: Robust scheduling features, including CPM and PERT.
- **Cost Management**: Comprehensive cost tracking and control.
- **Resource Management**: Advanced resource planning and optimization.
- **Libre Project**: Another example of a project management tool, often open-source, that supports various project management activities.

These tools vary in their licensing (open-source vs. commercial) and the depth of features they offer, but all aim to streamline the planning, execution, and monitoring of software projects.

Key Takeaways

- [ ] Configuration management is crucial for maintaining order and integrity in evolving software projects.
- [ ] The two main activities of configuration management are identification (deciding what to control) and control (managing changes).
- [ ] Work products are categorized as controlled, pre-controlled, or uncontrolled, with critical items requiring formal change procedures.
- [ ] Configuration control ensures only authorized changes are made, preventing unauthorized modifications to critical components.
- [ ] The "reserve" and "restore" operations are fundamental to the change workflow, allowing developers to work on private copies and integrate approved changes.
- [ ] The Change Control Board (CCB) plays a vital role in reviewing, validating, and approving all proposed changes before they are integrated.
- [ ] Release management streamlines the delivery of new software versions, minimizing effort for both developers and users.
- [ ] Tools like SCCS and RCS efficiently manage text file versions by storing only "deltas" (changes) rather than full copies.

- [ ] Various project management tools (e.g., Gantt Project, Microsoft Project, Primavera) offer features for scheduling, resource, cost, and portfolio management.

Conclusion

The detailed exploration of software configuration management underscores its indispensable role in modern software development. From the meticulous identification of critical work products to the stringent control over every modification, SCM provides the scaffolding necessary to build and maintain complex systems. The "reserve" and "restore" operations, coupled with the vigilant oversight of a Change Control Board, form a robust mechanism that ensures integrity, prevents conflicts, and promotes authorized, well-considered changes.

Furthermore, the discussion of release management highlights the importance of streamlining the delivery of software to end-users, a process that must be as efficient for developers as it is seamless for consumers. The introduction to tools like SCCS, RCS, Gantt Project, Microsoft Project, and Primavera illustrates the practical application of these theoretical concepts, demonstrating how specialized software can automate and enhance project oversight. Ultimately, mastering configuration management and leveraging appropriate project management tools are not just about technical proficiency; they are about orchestrating collaboration, mitigating risks, and ensuring that software projects evolve predictably and successfully, delivering value consistently over time.